南開大學

自然语言处理课程实验报告



实验名称：作业一：文本分类实现

姓　　名：张书晗

学　　号：2413157

专　　业：物联网工程

学　　院：网络空间安全学院

提交日期：2026 年 10 月 7 日

目录

1 实验目的 **3**

2 实验要求及要点 **3**

2.1 数据划分 3

2.2 评价指标 3

3 实验内容 **3**

3.1 Task 1：Bag of Words 3

3.1.1 代码说明 3

3.1.2 实现过程 4

3.1.3 Binary Bag of Words 5

3.1.4 Word Frequency 5

3.1.5 Task 1 总结 5

3.2 Task 2：Word2Vec / Word Embedding 6

3.2.1 代码说明 6

3.2.2 实现过程 6

3.2.3 Pre-trained GloVe（glove.6B，100d） 7

3.2.4 Word2Vec Trained on AG News 7

3.2.5 Word2Vec Trained on NYT 7

3.2.6 Task 2 总结 8

3.3 Task 3：预训练语言模型 8

3.3.1 代码说明 8

3.3.2 微调设置 9

3.3.3 实现过程 9

3.3.4 结果与分析 10

4 总结与结论 **10**

1 实验目的

围绕新闻文本分类任务，比较不同文本表示方法在相同数据划分与评价指标下的表现：

1. **Bag-of-Words**（Binary BoW、Word Frequency）+ Logistic Regression；
2. **静态词向量**（预训练 GloVe、在 AG News / NYT 上自训练 Word2Vec）经平均池化得到文档向量后 + Logistic Regression；
3. **预训练语言模型** BERT-base-uncased Fine-tuning。

2 实验要求及要点

- **NYT 新闻数据集**（主分类数据）
- **AG News 数据集**（用于训练 Word2Vec）

2.1 数据划分

对 NYT 使用 `random_state=42` 随机打乱后按 **80% / 10% / 10%** 划分为 Train / Validation / Test。

2.2 评价指标

- **Accuracy**：预测正确样本数 / 测试样本总数。
- **Macro-F1**：先对每个类别计算 F1，再对类别取算术平均。

3 实验内容

3.1 Task 1：Bag of Words

3.1.1 代码说明

Task 1 用 Binary BoW 与 Word Frequency 两种表示统一训练 Logistic Regression。

文件	作用
scripts/prepare_data.py	打乱 NYT 并做 80/10/10 划分
scripts/run_task1.py	Task 1 入口脚本
src/data/split.py	加载 / 划分 / 保存数据
src/features/bow.py	Binary BoW 与 Word Frequency 特征
src/models/classifiers.py	Logistic Regression 分类器
src/utils/tokenize.py	分词：小写 + nltk.word_tokenize
src/utils/metrics.py	Accuracy / Macro-F1
src/utils/io.py	指标写 JSON

3.1.2 实现过程

1. **准备数据**：运行 python scripts/prepare_data.py，从 data/raw/nyt.csv 读入 11519 条样本，random_state=42 打乱后按 80/10/10 划分，真实输出为：

```
Total samples: 11519
Train/Val/Test: 9215 / 1151 / 1153
```

   标签共 3 类：business（1429 条）/ politics（1451 条）/ sports（8639 条），这里要注意的是：sports 类明显占大头，类别明显不均衡——这也是我们统一用 class_weight="balanced" 的原因。还要注意：Task 1–3 共用这一份划分。

2. **构建词表**：我们在**训练集**上 fit 出词表（避免用 Test 信息），实测词表大小 $|\mathbf{V}| = \mathbf{127767}$。分词方式是：先文本小写，再 nltk.word_tokenize。通过自定义 analyzer 传给 CountVectorizer。对 9215 条训练文本做 fit 实测约 **11.6 秒**。

3. **训练与评估**：分类器统一为 LogisticRegression(solver="saga",max_iter=3000,class_weight="balanced",random_state=42)（saga 同时支持稀疏特征与 class_weight）。特征矩阵保持稀疏、不转稠密，因为我的电脑内存比较小。整条 python scripts/run_task1.py 跑完约 **117 秒**，耗时主要集中在 saga 求解器在高维稀疏特征上迭代。

4. 一个值得记录的观察：Task 1 的 saga 求解器在 3000 次迭代内**收敛、无告警**；而后面 Task 2（100 维稠密词向量）却触发了 sklearn 的 ConvergenceWarning（max_iter 用尽、系数未收敛），说明同一求解器在稠密、低维特征上反而更难在给定迭代数内收敛（详见第 4 章，我再详细说明）。

3.1.3 Binary Bag of Words

文档向量维度为词表大小 |V|：第 i 个词出现则为 1、否则为 0，不统计词频。我们用 `CountVectorizer(analyzer=..., binary=True)` 实现。

NYT Test 结果

Accuracy	Macro-F1
0.9879	0.9725

3.1.4 Word Frequency

文档向量同样基于训练词表，但第 i 维取该词在文档中的出现次数 c_i（`binary=False`）。

NYT Test 结果

Accuracy	Macro-F1
0.9905	0.9783

3.1.5 Task 1 总结

方法	Accuracy	Macro-F1
Binary BoW + LR	0.9879	0.9725
Word Frequency + LR	0.9905	0.9783

我做了两次实验，两次得到的数值完全一致（原因是词表构建与 LR 都固定了 `random_state`，nltk 分词也是确定性的）。Word Frequency 略优于 Binary，可能是因为词频信息在新闻主题词上有一定区分度。两类 BoW 在本数据上已达到 99% 的 Accuracy，说明类别主题词区分度高，配合 `class_weight="balanced"` 的 LR 拟合能力强；少数类 business 和 politics 上的表现将在第 6 章用 Macro-F1 进一步分析。

结果文件：`results/task1_binary.json`、`results/task1_count.json`。

3.2 Task 2：Word2Vec / Word Embedding

3.2.1 代码说明

文件	作用	说明
`scripts/run_task2.py`	Task 2 入口脚本	
`src/train/task2_word2vec.py`	GloVe / W2V(AG) / W2V(NYT) 三组流程	
`src/features/embeddings.py`	加载词向量、训练 Word2Vec、文档平均池化	封装 `gensim.Word2Vec`、`gensim.downloader`
`src/models/classifiers.py`	Logistic Regression	同 Task 1，封装 sklearn
`src/utils/tokenize.py`	分词	同 Task 1

引用的第三方资源：预训练 GloVe 来自 Stanford 的 `glove.6B.zip`；Word2Vec 训练复用 `gensim.models.Word2Vec`。

3.2.2 实现过程

1. **先文档编码**（`docs_to_avg_vectors`）：先和刚才 Task 1 一样，对每条文本小写、`nltk.word_tokenize` 分词，逐词查词向量，对命中词的向量取算术平均得到 100 维文档向量；OOV 词跳过，若整篇无有效词则用零向量。分类器设置与 Task 1 一致。

2. **GloVe 加载**：输出为：

```
Local GloVe file not found; downloading glove-wiki-gigaword-100 via gensim
    ...
```

3. **W2V(AG)**：读入 `data/raw/ag.csv` 全部 90000 条 `text`，逐条分词后交给 `gensim.models.Word2Vec` 训练（`vector_size=100,window=5,min_count=2,sg=1,epochs=5,seed=42,workers=4`），保存 `models/w2v_ag.model`，再用它编码 NYT。实测 AG 词表在 NYT 上的 OOV 率为 **4.51%**。

4. **W2V(NYT)**：**只用 NYT Train** 分词训练 Word2Vec，超参与 AG 相同，保存 `models/w2v_nyt.model`。实测 NYT 词表在 NYT 上的 OOV 率仅 **1.10%**，这说明语料更小但同域，词表覆盖反而最好。

5. **训练日志里的两个真实告警**：

- 训练 LR 时出现 sklearn 的 `ConvergenceWarning: saga` 在 `max_iter=3000` 内未完全收敛（`coef_ did not converge`）。这与 Task 1 形成对比：稠密低维词向量特征在该迭代数下未收敛，是后续可调的点，如加大 `max_iter`。
- 进程退出时打印多条 `Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'`，经了解后知道：这是 gensim 与 numpy 版本组合下解释器退出阶段的告警，不影响模型与指标。

3.2.3 Pre-trained GloVe（glove.6B，100d）

使用 Stanford GloVe 6B 100 维词向量（词表 40 万），平均池化后训练 LR。

NYT Test 结果

Accuracy	Macro-F1
0.9835	0.9694

3.2.4 Word2Vec Trained on AG News

在 AG News 90000 条文本上用 Skip-gram 训练 100 维 Word2Vec，再对 NYT 平均池化 + LR。

NYT Test 结果

Accuracy	Macro-F1
0.9679	0.9394

3.2.5 Word2Vec Trained on NYT

仅用 NYT Train 训练 100 维 Word2Vec，超参与 AG 一致，再对 NYT 平均池化 + LR。

NYT Test 结果

Accuracy	Macro-F1
0.9792	0.9560

3.2.6 Task 2 总结

方法	Accuracy	Macro-F1
GloVe 100d + LR	0.9835	0.9694
W2V(AG) 100d + LR	0.9679	0.9394
W2V(NYT) 100d + LR	0.9792	0.9560

关于可复现性的说明：GloVe 我的两次运行完全一致，但 **gensim Word2Vec 多线程训练存在 run-to-run 随机性**，尽管设了 `seed=42`，`workers=4` 的负采样更新顺序仍会受线程调度影响，我们前后两次运行得到的 W2V 指标有一点点浮动：W2V(AG) 的 Macro-F1 在 0.937 与 0.939 之间、W2V(NYT) 在 0.956 与 0.959 之间。

结论：预训练 GloVe 最好，因为大规模通用语料可以带来更稳的语义空间，同域自训 W2V(NYT) 优于跨域 W2V(AG)，原因：虽然 AG 语料远多于 NYT Train，但 AG 词表在 NYT 上的 OOV 率达 4.51%、而 NYT 词表仅 1.10%，说明**领域匹配比单纯扩大语料更重要**。

三者整体都略低于 Task 1 的高维 BoW。

结果文件：`results/task2_glove.json`、`results/task2_w2v_ag.json`、`results/task2_w2v_nyt.json`。

3.3 Task 3：预训练语言模型

3.3.1 代码说明

文件	作用	说明
`scripts/run_task3.py`	Task 3 入口脚本	
`src/train/task3_bert.py`	BERT 微调 → Test 评估流程	
`src/models/bert_classifier.py`	`TextDataset`、标签映射、tokenize、训练与预测	封装 `transformers` 的 `AutoTokenizer` / `AutoModelForSequenceClassification` / `Trainer` / `TrainingArguments`

3.3.2 微调设置

配置项	取值
预训练模型	`google-bert/bert-base-uncased`
batch size	16
learning rate	2e-5
优化器	AdamW
随机种子	42

3.3.3 实现过程

1. **启动训练**：运行 `PYTORCH_ENABLE_MPS_FALLBACK=1pythonscripts/run_task3.py`（`src/train/task3_bert.py`）。
2. **标签映射**：从 Train 标签按类别名排序构建 `label2id` / `id2label`（business / politics / sports 分别对应 0/1/2）。
3. **加载预训练**：`AutoTokenizer` + `AutoModelForSequenceClassification.from_pretrained("google-bert/bert-base-uncased",num_labels=3)`。加载时 transformers 打印了 `MISSING` 提示，这是因为 `num_labels=3` 使分类头参数随机初始化，分类头需要在下游任务从零训练，我们据此确认预训练主体权重加载正确。
4. **编码数据**：对 Train / Val / Test 文本 `truncation=True,padding=True,max_length=64`。这里有一个关键背景：NYT 文档 `max_length=64` ，意味着只保留开头一部分的正文，其余被截断，这会直接影响后文分析。
5. **Trainer 微调**：`num_train_epochs=3,per_device_train_batch_size=16,learning_rate=2e-5,eval_strategy="epoch",seed=42`；检测到 MPS 后 `model.to("mps")`、关闭 fp16。
6. **我开了训练监控**，结果：每个 epoch 576 步、共 1728 步，MPS 上约 1.5–1.85 it/s。训练 loss 单调下降：epoch 1 约 0.128→0.094，epoch 2 约 0.068→0.034，epoch 3 约 0.006→0.009；learning rate 按线性从 2e-5 衰减到接近 0。每个 epoch 结束后在 Validation 上 eval。
7. **保存与 Test 评估**：训练结束后把最终权重写入 `models/bert/final/`，再在 Test 上按 batch=32 前向推理取 `argmax` 得到标签，计算 Accuracy / Macro-F1 。

3.3.4 结果与分析

NYT Test 结果

Accuracy	Macro-F1
0.9809	0.9573

关于可复现性的说明：BERT 在 MPS 上的微调我两次运行得到的 Test 指标从 0.9775 / 0.9515 变为 0.9809 / 0.9573，这应该属于随机波动。

分析：BERT 低于 Task 1 的强 BoW 基线，与预训练 GloVe 接近，高于两个自训练 Word2Vec。BERT 会随运行浮动，这说明在 3 epochs、`max_length=64` 的固定预算下，不宜直接断言 BERT 一定不如静态词向量。限制 BERT 发挥的主要因素是 `max_length=64` 截断了大部分正文，还有三类新闻主题词区分度高，高维稀疏 BoW + LR 已足够强。BERT 仍达到 98% 的 Accuracy，说明我们的微调流程正确，其优势更可能体现在需要上下文消歧的细粒度任务上。

结果文件：`results/task3_bert.json`。

4 总结与结论

1. 通过完整做完了本次实验，我熟悉了文本分类任务的流程：数据划分 → 文本表示 → 分类器 → Test 上的 Accuracy / Macro-F1，也更熟悉了三种方法的区别：BoW 是高维稀疏，数有没有出现以及出现几次；而 GloVe / Word2Vec 是先变成稠密词向量，再对文档做平均；而 BERT 是带上下文的预训练语言模型，可以在下游分类任务上做 Fine-tuning。

2. 我更深刻理解了，文本分类任务中，评价指标不能只看 Accuracy，就比如本次实验中，因为 NYT 里 sports 类别占绝大多数，Accuracy 很容易虚高，而 Macro-F1 更能看出 business 和 politics 这些少数类有没有被照顾到。

3. 在主题词很明显、类别区分度高的新闻分类场景里，可以通过高维 BoW 以及带类别权重的 Logistic Regression 实现，效果应该会很好。而在缺少标注、又想快速得到语义向量的场景里，可以通过预训练 GloVe 或同域自训 Word2Vec，配合平均池化，但要注意，平均池化会丢掉词序，在复杂语义场景中可能不够用。而在更依赖上下文的任务里，BERT 是更合适的。